

用Dapper存取資料庫

聖殿祭司。奚江華

Dapper概観

Dapper參考網站

- ✦ Github - Dapper

<https://github.com/StackExchange/Dapper>

- ✦ Dapper Tutorial

<https://dapper-tutorial.net/>

What's Dapper ?

- ✦ Dapper is a Micro ORM
- ✦ Dapper特色：體積小、執行速度快
- ✦ 需明確宣告SQL命令字串（和資料庫有相依）
- ✦ Entity Framewor抽象化後端資料庫平台（和資料庫無相依）

Dapper和EF有什麼不同？

- Dapper沒有DbContext，亦即沒有DbContext作為對資料庫的中樞
- 故也不需在DbContext註冊DbSet<T> Data Model
- 不支援EF Code First Migration
- Data Model對Dapper不是必要，支援dynamic type
- Dapper的CRUD語法比較像是直傳SQL Statement給資料庫；而EF則較常用LINQ或ORM操作方式，後續Provider再轉換成特定資料庫語法
- MVC的Scaffolding只支援EF DbContext及Data Model的模式
- Dapper效能比EF快

Dapper和EF是二選一或協同？

- 技術上，一個專案可混用Dapper和EF。至於實務上混不混用，看個人或公司理念與想法
- 可利用EF Migrations負責對資料庫創建、異動、同步與種子資料。讓Dapper負責對資料庫的CRUD存取，取用Dapper的直傳SQL語法及較快效能
- Dapper沒有從資料庫Scaffolding出Data Model資料模型的功能，而EF有
- Dapper和EF語法皆實作在Repository較佳

使用Dapper三步驟

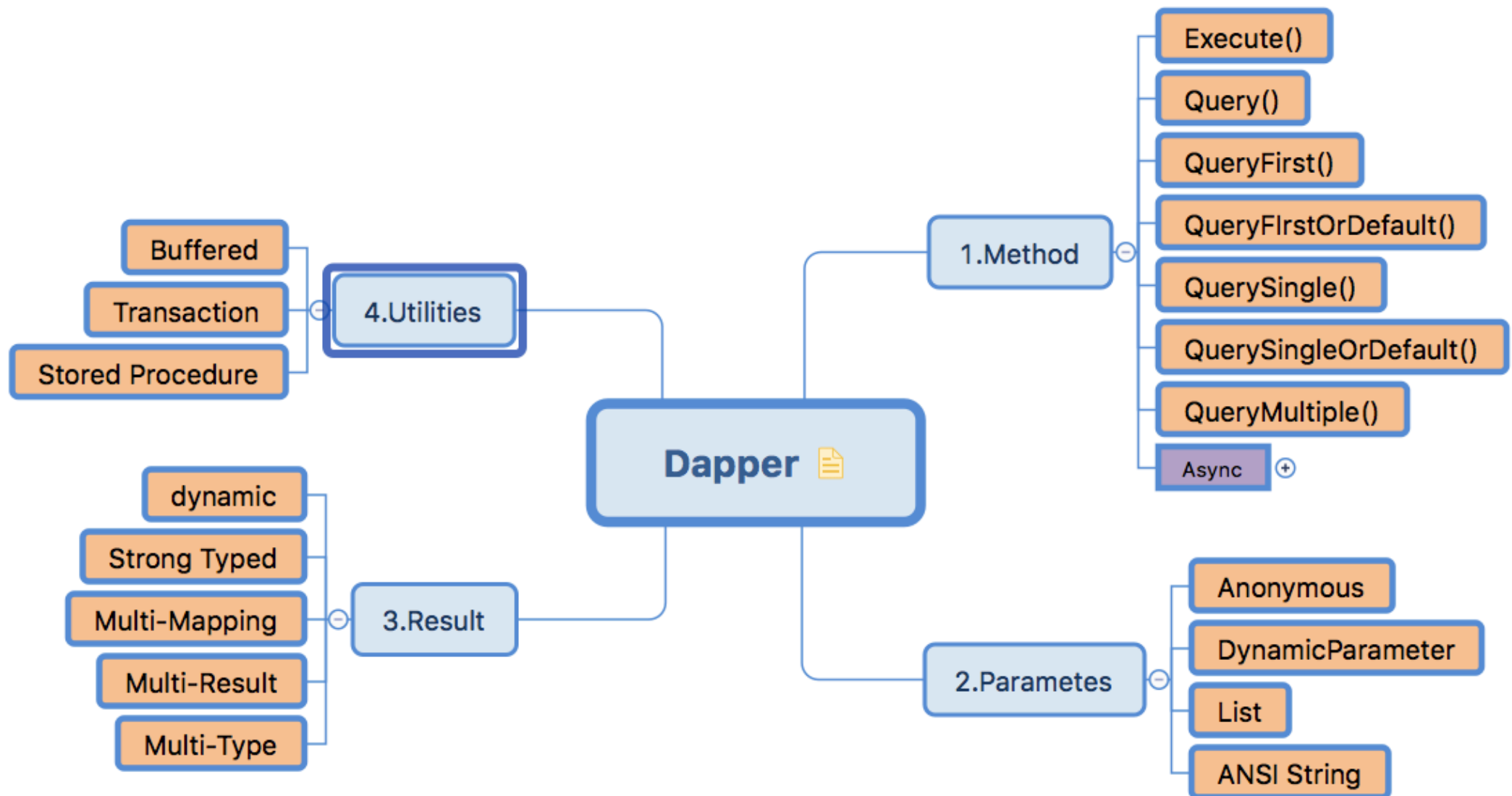
- ① 建立資料庫連線物件
- ② 建立SQL命令字串
- ③ 將SQL命令字串傳遞給Query(sql)等方法作查詢

安裝ConfigurationManage套件

用NuGet安裝System.Configuration.ConfigurationManage套件，用來存取App.config或Web.Config中資料庫連線字串

```
static string connString = ConfigurationManager.  
    ConnectionStrings["NorthwindContext"].ConnectionString;
```


從四個面向理解Dapper



九種執行方法

Dapper內建九種方法

- ✦ Execute
- ✦ ExecuteReader
- ✦ ExecuteScalar
- ✦ Query
- ✦ QueryFirst
- ✦ QueryFirstOrDefault
- ✦ QuerySingle
- ✦ QuerySingleOrDefault
- ✦ QueryMultiple

查詢並回傳結果之方法

- ✦ Query()
- ✦ QueryFirst()
- ✦ QueryFirstOrDefault()
- ✦ QuerySingle()
- ✦ QuerySingleOrDefault()
- ✦ QueryMultiple() 在同一命令執行多次查詢

Execute方法

- 執行命令一或多次，會回傳受影響Rows數目
- 通常用於執行：
 - 預存程序
 - Insert陳述式
 - Update陳述式
 - Delete陳述式

Method讀取/執行方法

Employee模型

```
public class Employee
{
    public int EmployeeID { get; set; }
    public string LastName { get; set; }
    public string Title { get; set; }
    public string City { get; set; }
    public string Country { get; set; }
}
```


1.Query<T>() - 回傳強型別

- ✱ 執行查詢，並回傳強型別物件

```
List<Employee> employees = null;
```

```
//Query查詢回傳強型別
```

```
using (SqlConnection conn = new SqlConnection(@"Data Source=..."))
```

```
{
```

```
    string sql = "select * from Employees";
```

```
    employees = conn.Query<Employee>(sql).ToList();
```

```
}
```

Query查詢

回傳Employee型別

```
foreach (var emp in employees)
```

```
{
```

```
    Console.WriteLine($"{emp.EmployeeID}, {emp}, {emp.Title}, {emp.City}, {emp.Country}");
```

```
}
```

顯示結果

2.Query() - 回傳dynamic List

- 執行查詢，並回傳一個dynamic動態型別物件

```
using (SqlConnection conn = new SqlConnection(@"Data Source=..."))
{
    string sql = "select * from Employees";
    var Employees = conn.Query(sql).ToList();

    foreach (var emp in Employees)
    {
        Console.WriteLine($"{emp.EmployeeID}, {emp.LastName}, {emp.Title}, {emp.City}, {emp.Country}");
    }
}
```

Query查詢

dynamic List

顯示結果

3.QueryFirstOrDefault()

- 執行查詢，並回第一筆資料(多筆回傳第一筆)

```
using (SqlConnection conn = new SqlConnection(@"Data Source=..."))
```

```
{
```

```
    string sql = "select * from Employees where Country='UK'";
```

```
    var emp = conn.QueryFirstOrDefault<Employee>(sql);
```

查詢第一筆資料

```
    Console.WriteLine($"{emp.EmployeeID}, {emp.LastName}, {emp.Title},  
        {emp.City}, {emp.Country}");
```

```
}
```


4.QuerySingleOrDefault()

- ✦ 查詢唯一一筆資料，若有多筆會擲出錯誤

```
using (SqlConnection conn = new SqlConnection(@"Data Source=..."))
```

```
{
```

```
    string sql = "select * from Employees where LastName='King'";
```

```
    var emp = conn.QuerySingleOrDefault<Employee>(sql);
```

查詢唯一一筆資料

```
    Console.WriteLine($"{emp.EmployeeID}, {emp.LastName}, {emp.Title},  
        {emp.City}, {emp.Country}");
```

```
}
```


5. 指定多重參數

```
static void QueryParameters()
{
    List<Employee> employees;
    using (SqlConnection conn = new SqlConnection(connString))
    {
        string sql = "select * from Employees where Country=@country AND
            TitleOfCourtesy=@titleOfCourtesy";
        employees = conn.Query<Employee>(sql, new { country="USA",
            titleOfCourtesy="Ms." }).ToList();
    }

    foreach (var emp in employees)
    {
        Console.WriteLine($"{emp.EmployeeID}, {emp.Title}, {emp.City}, {emp.Country},
            {emp.TitleOfCourtesy}");
    }
}
```

指定多重參數

6. 傳遞參數集合作查詢

```
static void QueryParameterList()
{
    List<Employee> employees = null;
    //Query1
    using (SqlConnection conn = new SqlConnection(connString))
    {
        string sql = "select * from Employees where EmployeeID in @IDs";
        employees = conn.Query<Employee>(sql, new { IDs = new int[]
            { 1, 3, 5 } }).ToList();
    }
    ...
}
```

傳遞參數集合

7. 查詢預存程序

- 查詢SQL Server資料庫Stored Procedure預存程序。呼叫FindEmployeeByName預存程序，並傳遞參數

```
static void QueryStoredProcedure()
```

```
{
```

```
using (SqlConnection conn = new SqlConnection(@"..."))
```

```
{
```

```
var emp = conn.Query<Employee>("FindEmployeeByName",
```

```
new { LastName = "King", FirstName = "Robert" },
```

```
commandType: CommandType.StoredProcedure
```

```
).SingleOrDefault();
```

```
Console.WriteLine($"{emp.EmployeeID}, {emp.LastName}, {emp.Title},  
{emp.City}, {emp.Country}");
```

```
}
```

```
}
```

預存程序

參數

指定命令類型

建立預存程序語法

以Northwind資料庫為例

Create Procedure **FindEmployeeByName**

預存程序名稱

@LastName nvarchar(20),

@FirstName nvarchar(10)

參數名稱及型別

AS

Select EmployeeID, LastName, FirstName, Title, City, Country

from dbo.Employees

where FirstName = **@FirstName** AND LastName = **@LastName**

go

SQL陳述式

8.QueryAsync非同步查詢

```
static void QueryAsyncStrongStyped()
{
    List<Employee> employees = null;
    //Query查詢回傳強型別
    using (SqlConnection conn = new SqlConnection(connString))
    {
        string sql = "select * from Employees";
        employees = conn.QueryAsync<Employee>(sql).Result.ToList();
    }
    ...
}
```


9.Execute()執行Insert

```
static int ExecuteInsert(Employee emp)
{
    int affectedRow = 0;
    using (SqlConnection conn = new SqlConnection(connString))
    {
        string sql = "INSERT INTO Employees(FirstName, LastName, Title, Country)
            VALUES ( @FirstName, @LastName, @Title, @Country)";
        affectedRow = conn.Execute(sql, new { emp.FirstName, emp.LastName,
            emp.Title, emp.Country });
    }

    return affectedRow;
}
```


10.Execute()執行Update

```
static int ExecuteUpdate(Employee emp)
{
    int affectedRow = 0;
    using (SqlConnection conn = new SqlConnection(connString))
    {
        string sql = "Update Employees Set FirstName=@FirstName, LastName=@LastName, Title=@Title, Country=@Country
            WHERE EmployeeID = @EmployeeID";
        affectedRow = conn.Execute(sql, new
        {
            FirstName = emp.FirstName,
            LastName = emp.LastName,
            Title = emp.Title,
            Country = emp.Country,
            EmployeeID = emp.EmployeeID
        });
    }

    return affectedRow;
}
```


11.Execute()執行Delete

```
static int ExecuteDelete(Employee emp)
{
    int affectedRow = 0;
    using (SqlConnection conn = new SqlConnection(connString))
    {
        string sql = "DELETE FROM Employees WHERE EmployeeID = @EmployeeID";
        affectedRow = conn.Execute(sql, new { emp.EmployeeID });
    }

    return affectedRow;
}
```


12. 資料庫交易-使用SqlTransaction

```
static void DBTransaction()
{
    string sql = "INSERT INTO Customers (CustomerName) Values (@CustomerName);";
    using (SqlConnection conn = new SqlConnection(connString))
    {
        conn.Open();
        using (SqlTransaction tran = conn.BeginTransaction())
        {
            var affectedRows = conn.Execute(sql, new { CustomerName = "Mark" },
                transaction: tran);

            tran.Commit();

            Console.WriteLine(affectedRows);
        }
        conn.close();
    }
}
```


13. 資料庫交易-使用TransactionScope

```
static void DBTransactionScope()
{
    string sql = "INSERT INTO Customers (CustomerName) Values (@CustomerName)";
    using (SqlConnection conn = new SqlConnection(connString))
    {
        conn.Open();
        using (TransactionScope transaction = new TransactionScope())
        {
            var affectedRows = conn.Execute(sql, new { CustomerName = "Mark" });
            transaction.Complete();
            Console.WriteLine(affectedRows);
        }
        conn.close();
    }
}
```


14.ExecuteReader()方法

將資料填入DataTable或DataSet

```
static void ExecuteReader()
{
    using (SqlConnection conn = new SqlConnection(connString))
    {
        string sql = "select * from Employees";
        var reader = conn.ExecuteReader(sql);

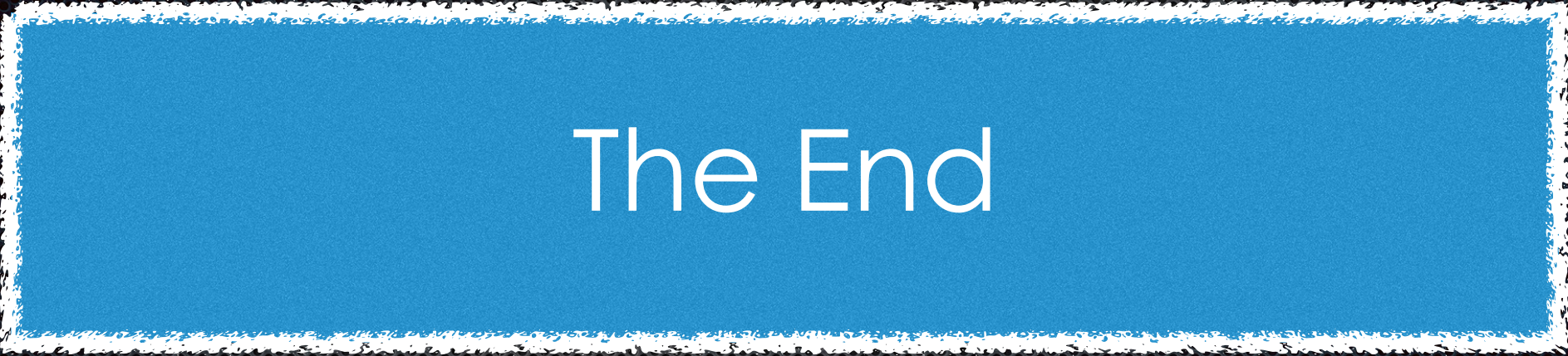
        DataTable dt = new DataTable();
        dt.Load(reader);
    }
}
```


15.ExecuteScalar()方法

回傳第一列的第一欄資料

```
static void ExecuteScalar()
{
    using (SqlConnection conn = new SqlConnection(connString))
    {
        string sql = "select FirstName from Employees where EmployeeId=1";
        string Name = conn.ExecuteScalar<string>(sql);

        Console.WriteLine(Name);
    }
}
```

The End

奚江華個人經歷簡介

- ✦ 著有ASP.NET 2.0、3.5、4.5、4.6等熱門暢銷書籍，以及**ASP.NET MVC 5.x範例完美演繹**
- ✦ 歷任台灣微軟MSDN、TechEd、TechDay研討會講師
- ✦ 曾數屆當選微軟MVP
- ✦ 國內各大上市公司及大專院校之培訓講師
- ✦ 曾發表文章刊於ITHOME電腦週刊

碼魔法FB & 網站

- ✦ CodeMagic碼魔法軟體學院Facebook

www.facebook.com/codemagictw

- ✦ CodeMagic碼魔法網站

www.codemagic.com.tw